UniSENAI

ATIVIDADE COMPLEMENTAR – CRUD COM PHP, BANCO DE DADOS E PROGRAMAÇÃO ORIENTADA A OBJETOS

1. Contextualização

Na primeira etapa desta atividade, você desenvolveu, de forma guiada, uma aplicação web para gerenciamento de contatos utilizando **PHP procedural e banco de dados MySQL**.

A aplicação desenvolvida possui as operações básicas de um CRUD:

- **Create** – cadastrar um contato;
- **Read** – listar os contatos cadastrados;
- **Update** – editar um contato existente;
- **Delete** – excluir um contato.

A aplicação utiliza o banco de dados agenda e a tabela contatos, composta pelos campos id, nome, telefone, email e data_cadastro.

Nesta segunda etapa, o objetivo é ir além da simples reprodução do código. Você deverá **reconstruir a mesma aplicação utilizando os conceitos de Programação Orientada a Objetos (POO) estudados em aula**.

---

2. Desafio

Utilizando como ponto de partida o CRUD desenvolvido anteriormente, **reescreva a aplicação utilizando Programação Orientada a Objetos**, mantendo as mesmas funcionalidades da versão procedural.

A aplicação deverá continuar permitindo:

1. Cadastrar contatos;
2. Listar contatos;
3. Editar contatos;
4. Excluir contatos;
5. Validar os dados informados;
6. Utilizar declarações preparadas nas operações com o banco de dados;
7. Confirmar a exclusão de um contato.

Importante: não se trata apenas de transformar funções em métodos.

O objetivo é **identificar responsabilidades e organizar o sistema de acordo com os princípios da Programação Orientada a Objetos estudados em aula.**

3. Requisitos da versão orientada a objetos

Ao reconstruir a aplicação, você deverá aplicar os conceitos de POO trabalhados nas diferentes versões da calculadora desenvolvida em aula.

3.1 Classes e objetos

Identifique as principais entidades e responsabilidades existentes no sistema e crie classes que representem essas responsabilidades.

Por exemplo, você deverá analisar a necessidade de classes responsáveis por:

- representar um contato;
- realizar a conexão com o banco de dados;
- realizar as operações relacionadas aos contatos;
- controlar o fluxo das requisições da aplicação.

A definição das classes deverá ser feita por você, a partir da análise do problema.

3.2 Atributos e métodos

As informações que representam um contato deverão ser organizadas como atributos de um objeto.

As operações realizadas sobre essas informações deverão ser representadas por métodos, evitando que toda a lógica da aplicação fique concentrada em arquivos PHP procedurais.

3.3 Encapsulamento

Utilize modificadores de acesso adequados (private, protected e public) para controlar o acesso aos atributos e métodos das classes.

Analise quais informações devem ser acessadas diretamente e quais devem ser protegidas dentro das classes.

3.4 Divisão de responsabilidades

A aplicação deverá respeitar o princípio trabalhado em aula:

> **Uma classe deve possuir uma responsabilidade bem definida e realizá-la da melhor maneira possível.**

Evite criar uma única classe responsável por:

- receber dados do formulário;
- validar dados;
- conectar ao banco;
- executar SQL;
- apresentar HTML;
- controlar toda a aplicação.

Analise como essas responsabilidades podem ser distribuídas entre diferentes classes.

3.5 Interface

Analise se existe alguma responsabilidade que possa ser definida por meio de uma **interface**.

Caso seja pertinente à estrutura desenvolvida, utilize uma interface para estabelecer um contrato que possa ser implementado por uma ou mais classes.

O objetivo não é utilizar uma interface simplesmente para cumprir uma exigência, mas compreender **quando e por que esse recurso pode ser utilizado em um projeto orientado a objetos.**

3.6 Herança e Polimorfismo

Analise a estrutura desenvolvida e identifique se existe alguma situação em que **herança e polimorfismo** possam ser utilizados de maneira coerente.

Caso exista uma aplicação adequada desses conceitos, implemente-os.

Caso você conclua que não existe uma relação de herança adequada para este problema, **justifique sua decisão**.

> O objetivo não é utilizar herança ou polimorfismo de forma artificial, mas compreender que esses recursos devem ser utilizados quando fizerem sentido para o projeto.

---

4. Organização dos arquivos

Na versão orientada a objetos, organize os arquivos de maneira diferente da versão procedural.

Utilize como referência o conceito de **MVC (Model – View – Controller)** trabalhado em aula.

Procure separar:

Model

- classes responsáveis pelos dados e pelas regras relacionadas ao domínio;
- classes responsáveis pelo acesso ao banco de dados.

View

- páginas responsáveis pela apresentação das informações ao usuário;
- formulários e tabelas HTML.

Controller

- responsáveis por receber as requisições;
- chamar os métodos apropriados;
- controlar o fluxo entre a View e o Model.

A estrutura final deverá demonstrar que você compreendeu a finalidade da separação de responsabilidades.

---

5. Banco de dados

Utilize o mesmo banco de dados e a mesma tabela da atividade anterior.

Banco de dados:

`agenda`

Tabela:

`contatos`

Campos:

- `id`
- `nome`
- `telefone`

- `email`
- `data_cadastro`

Não é necessário criar uma nova estrutura de banco de dados.

A proposta é justamente perceber que **o banco de dados pode permanecer o mesmo, enquanto a arquitetura e a organização do código são modificadas.**

---

6. Comparação entre as duas versões

Após finalizar a versão orientada a objetos, produza uma breve análise comparando as duas implementações.

Responda:

1. Qual é a principal diferença entre a versão procedural e a versão orientada a objetos?
2. Quais responsabilidades estavam concentradas nos arquivos da versão procedural?
3. Como essas responsabilidades foram distribuídas na versão POO?
4. Quais classes foram criadas e qual é a responsabilidade de cada uma?
5. Onde foram utilizados atributos e métodos?
6. Como foi aplicado o encapsulamento?
7. Como a organização MVC contribuiu para estruturar o projeto?
8. Foi utilizada interface? Se sim, qual foi sua finalidade? Se não, justifique.
9. Foi utilizada herança? Se sim, qual foi sua finalidade? Se não, justifique.
10. Na sua opinião, qual das duas versões apresenta melhor organização e por quê?

---

7. Critérios de avaliação

A avaliação considerará principalmente a **compreensão e aplicação dos conceitos de POO**, e não apenas o funcionamento da aplicação.

Serão considerados:

- funcionamento correto do CRUD;
- utilização adequada de classes e objetos;
- utilização de atributos e métodos;
- encapsulamento;
- divisão adequada de responsabilidades;
- organização dos arquivos;

- utilização coerente do padrão MVC;
- utilização adequada de interface, quando aplicável;
- utilização adequada de herança e polimorfismo, quando aplicável;
- qualidade e organização do código;
- capacidade de explicar as decisões tomadas durante a implementação.

Atenção

Não será suficiente criar classes apenas para "transformar" o código procedural em POO.

Durante a apresentação da atividade, você deverá ser capaz de **explicar a responsabilidade de cada classe, o motivo da criação dos métodos e como ocorre a comunicação entre as diferentes partes da aplicação.**

O objetivo principal desta etapa é demonstrar que você compreendeu que **Programação Orientada a Objetos não significa apenas utilizar classes**, mas organizar o software a partir de objetos, responsabilidades e relações entre seus componentes.

---

8. Entrega

Entregue o projeto completo contendo:

- arquivos PHP;
- classes;
- organização MVC;
- arquivos HTML/CSS/JavaScript utilizados;
- script do banco de dados;
- demais arquivos necessários para execução do sistema.

A aplicação deverá ser executada em ambiente local, como XAMPP, e deverá permitir realizar todas as operações do CRUD.

Ao final, esteja preparado para apresentar o projeto e explicar **como a versão POO foi construída a partir da versão procedural.**

Objetivo final

Ao concluir esta atividade, você deverá ser capaz de perceber, na prática, a diferença entre:

"fazer o sistema funcionar"

e

“organizar o sistema para que ele seja compreensível, reutilizável, manutenível e dividido em responsabilidades”.

Esta é a principal mudança de perspectiva proporcionada pela Programação Orientada a Objetos.